Introduction to Algorithms

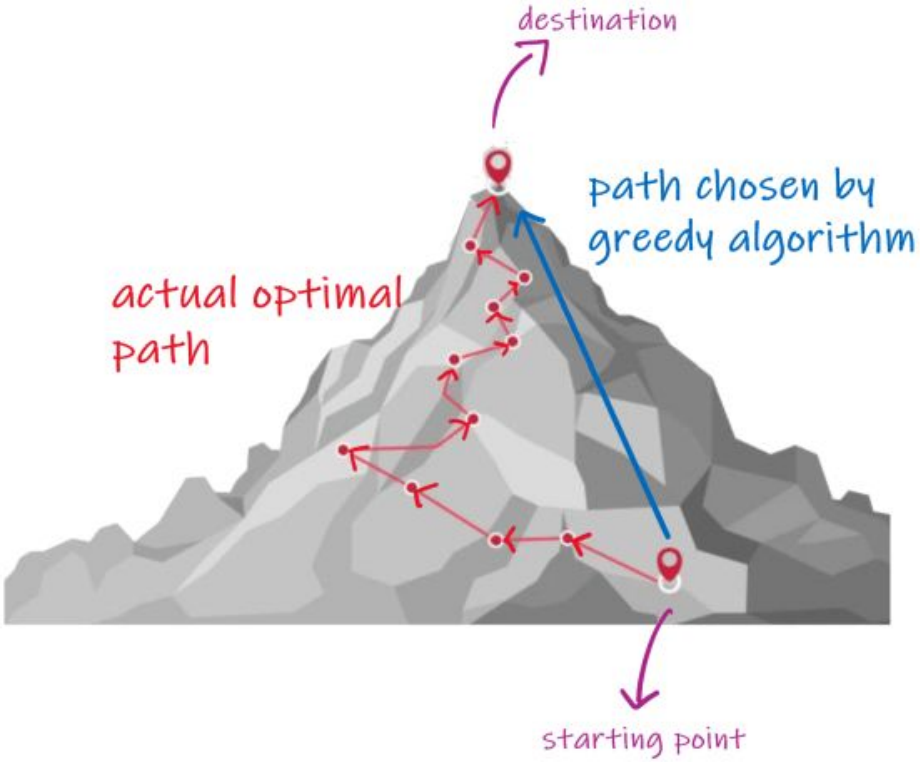
Levon Prazyan

Course Overview

- **Introduction**
- **Sorting and Searching**
- **Arrays and Lists**
- **Stacks and Queues, Double Ended Queue**
- **Heaps**
- **Trees**
- **Hash Tables**
- **Dynamic Programming**
- **Greedy Algorithms**
- **Graph Algorithms**
- **Network Flow**
- **NP-Completeness**

Greedy Algorithms

- A **greedy algorithm** is a strategy for solving optimization problems by **making the best possible (locally optimal) choice at each step**, with the hope that these local choices lead to a **globally optimal solution**.
- Greedy algorithms do not always yield optimal solutions, but for many problems they do.



Greedy Approach: Main Characteristics (1)

1. Local Optimal Choice:
   At each step, the algorithm chooses the option that seems best at that moment.
   - No backtracking
   - No reconsidering past decisions
2. Irrevocable Decisions:
   Once a choice is made, it is never changed.
   This makes greedy algorithms:
   1. Fast
   2. Simple to implement
      But sometimes risky if the problem doesn’t support greedy behavior

Greedy Approach: Main Characteristics (2)

3. Efficiency:
   Greedy algorithms are typically:
   - Time-efficient (often $O(n)$ or $O(n\log n)$)
   - Use little memory
4. Not Always Optimal:
   Greedy works only for specific problems.
   For some problems, greedy choices can lead to suboptimal results.

Greddy Algorithms and DP

Aspect	Greedy	Dynamic Programming
Decision making	Local	Global
Revisits decisions	No	Yes
Complexity	Lower	Higher
Optimality	Not guaranteed	Guaranteed (if applicable)

If you’re unsure whether greedy works ⟶
it's often safer to think in terms of **dynamic programming**.

Activity Selection Problem (1)

Select maximum number of non-overlapping activities.

- A set $S = \{a_1, a_2, \dots, a_n\}$ n proposed activities that wish to reserve the conference room, and the room can serve only one activity at a time.
- Each activity a_i has a start time s_i and a finish time f_i ; $0 \leq s_i < f_i \leq \infty$.
- If selected, activity a_i takes place during the half-open time interval $[s_i, f_i)$.
- Activities a_i and a_j are ***compatible*** if the intervals $[s_i, f_i)$ and $[s_j, f_j)$ don't overlap.
- Assume $f_1 \leq f_2 \leq \cdots \leq f_n$

Activity Selection Problem (2)

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	7	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

- A set $\{a_1, a_2, \ldots, a_{11}\}$ of activities.
  Activity a_i has start time s_i and f_i finish time.
- The subset $\{a_2, a_7, a_{11}\}$ is mutually compatible subset
- The subset $\{a_1, a_4, a_9, a_{11}\}$ is the maximum subset (not only).

Activity Selection Problem: Optimal Substructure

- Donate $S_{ij} = \{a_r \mid s_r \geq f_i \text{ and } f_r \leq s_j\}$
- Donate A_{ij} a maximum set of mutually compatible activities in S_{ij}
- Suppose $a_k \in A_{ij}$
  - Let: $A_{ik} = A_{ij} \cap S_{ik}$ and $A_{kj} = A_{ij} \cap S_{kj}$
  - So: $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$
  - $|A_{ij}| = |A_{ik}| + 1 + |A_{kj}|$
  - A_{ij} must also include optimal solutions to the two subproblems for S_{ik} and S_{kj} otherwise A_{ij} will not be the solution of S_{ij}

Activity Selection Problem: Dynamic Programming

- dp[i] - maximum number of mutually compatible activities ending at s_i
- $dp[i] = 1 + \max(dp[j]);\ j < i \text{ and } f_j \leq s_i$

i	Activity (s, f)	Compatible Prev j	dp[i] calculation	dp[i]
1	(1, 4)	-	1	1
2	(3, 5)	-	1	1
3	(1, 6)	-	1	1
4	(5, 7)	1, 2	1 + max(1, 1)	2
5	(5, 8)	1, 2	1 + max(1, 1)	2
6	(3, 9)	-	1	1
7	(6, 10)	1, 2, 3	1 + max(1, 1, 1)	2
8	(8, 11)	1, 2, 3, 4, 5	1 + max(1, 1, 1, 2, 2)	3

Activity Selection Problem: Greedy Choice (1)

1. Choose an activity that leaves the resource available for as many other activities as possible.
2. Of the activities choose one that finishes earliest – choose a_1.
3. Only one subproblem remains: finding activities that start after a_1 finishes.
4. Let $S_k = \{a_i \in S: s_i \geq f_k\}$:
5. S_1 is the only remaining subproblem after choosing a_1.

Is the greedy choice correct ?

Activity Selection Problem: Greedy Choice (2)

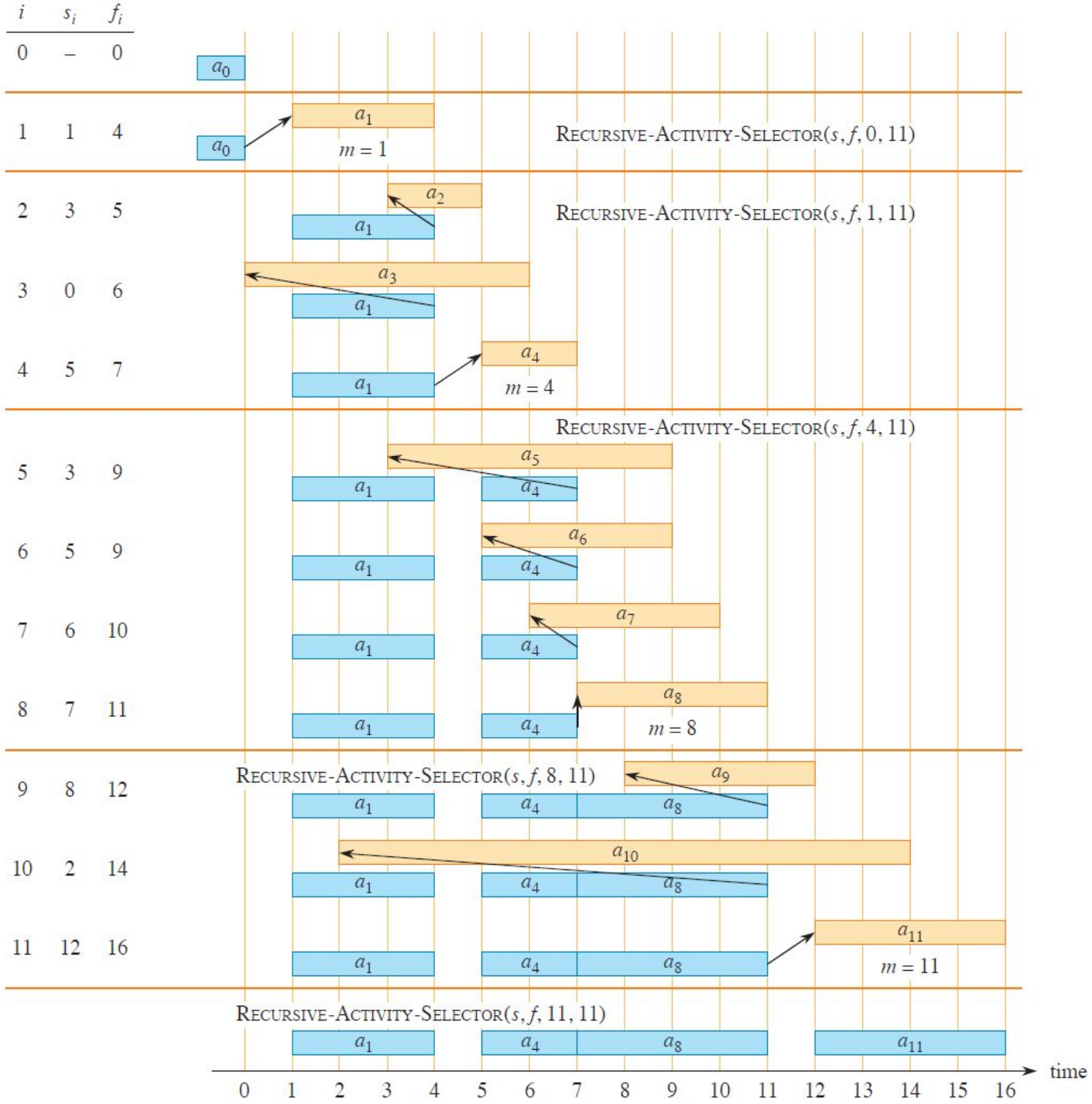
- **Theorem:** Consider any nonempty subproblem S_k and let $a_m \in S_k$ with the earliest finish time. Then a_m is included in some maximum-size subset of mutually compatible activities of S_k.
- **Proof:**
  - Let A_k be a max-size subset of mutually compatible activities in S_k and let $a_j \in S_k$ with earliest finish time.
  - If $a_j \neq a_m$, let the $A'_k = (A_k - \{a_j\}) \cup \{a_m\}$
  - A'_k elements are mutually compatible and $|A'_k| = |A_k|$
  - A'_k is a max-size subset of mutually compatible activities of S_k and includes a_m

Activity Selection Problem: Recursive Greedy Algorithm

```
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)
1  m = k + 1
2  while m ≤ n and s[m] < f[k]        // find the first activity in S_k to finish
3        m = m + 1
4  if m ≤ n
5        return {a_m} ∪ RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n)
6  else return ∅
```

- Add a_0 with $f_0 = 0$
- RECURSIVE-ACTIVITY-SELECTOR (s, f, 0, n)

Activity Selection Problem: Recursive Greedy Algorithm Example



Activity Selection Problem: Iterative Greedy Algorithm

```
GREEDY-ACTIVITY-SELECTOR(s, f, n)
1  A = {a_1}
2  k = 1
3  for m = 2 to n
4       if s[m] ≥ f[k]          // is a_m in S_k?
5            A = A ∪ {a_m}      // yes, so choose it
6            k = m              // and continue from there
7  return A
```

Elements of Greedy Strategy (1)

1. Determine the optimal substructure of the problem.
2. Develop a recursive solution.
3. Show that if you make the greedy choice, then only one subproblem remains.
4. Prove that it is always safe to make the greedy choice.
5. Develop a recursive algorithm that implements the greedy strategy.
6. Convert the recursive algorithm to an iterative algorithm.

Elements of Greedy Strategy (2)

1. Cast the optimization problem as one in which you make a choice and are left with one subproblem to solve.
2. Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.
3. Demonstrate optimal substructure by showing that, having made the greedy choice, what remains is a subproblem with the property that if you combine an optimal solution to the subproblem with the greedy choice you have made, you arrive at an optimal solution to the original problem.

Elements of Greedy Strategy (3)

How can you tell whether a greedy algorithm will solve a particular optimization problem?

No way works all the time, but

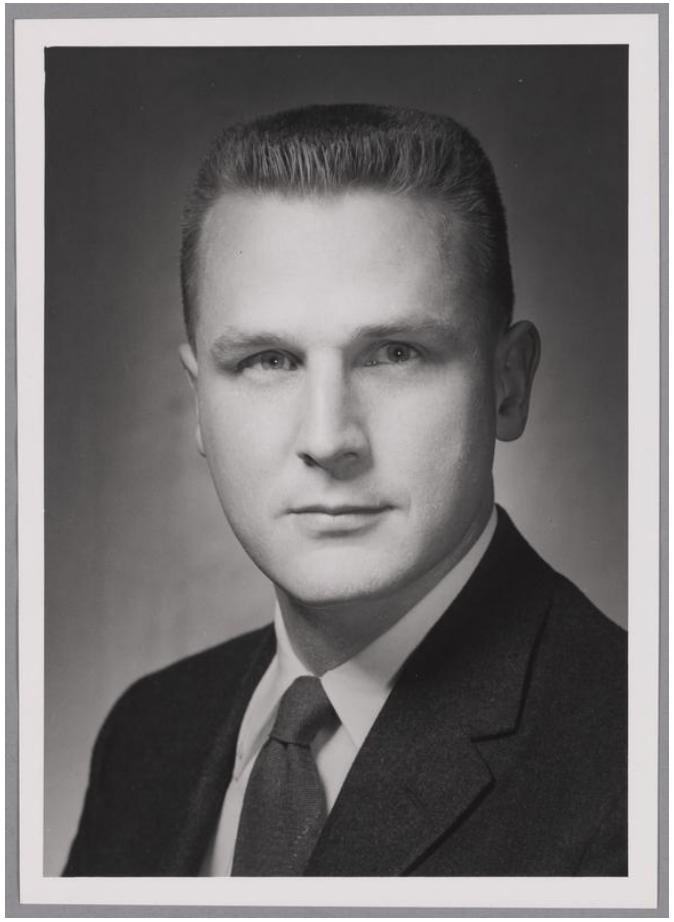
1. greedy-choise property and
2. optimal substructure

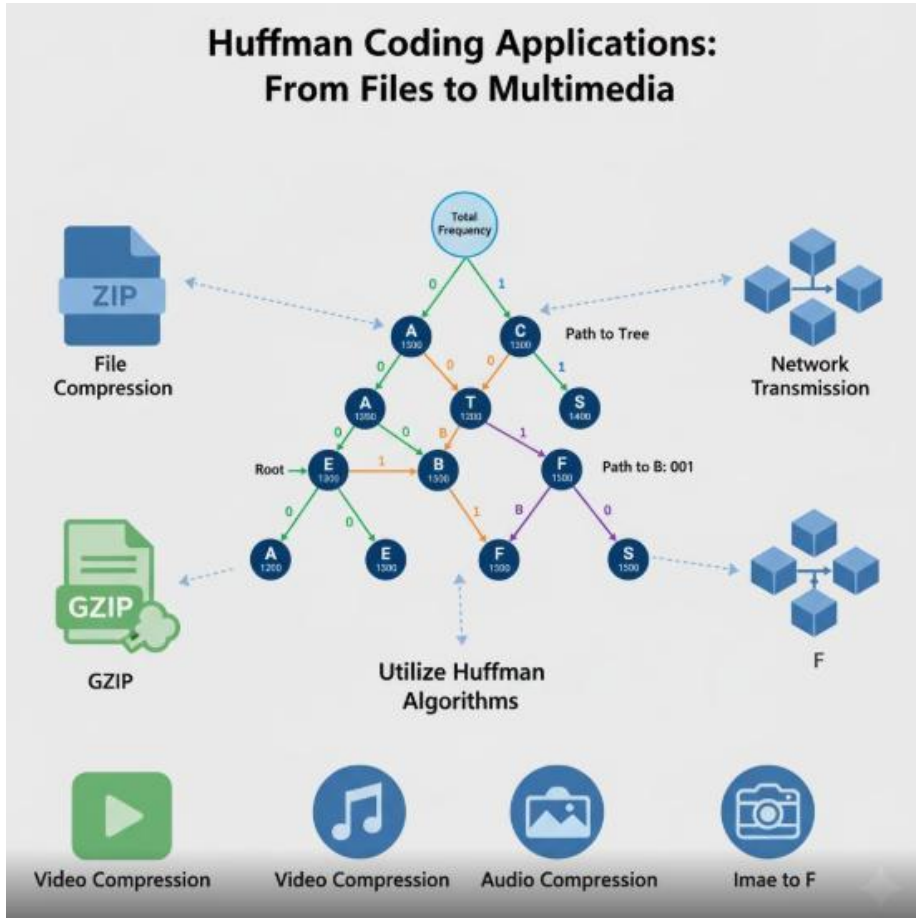
are two key ingredients.

Greedy Approach vs Dynamic Programming

Feature	Greedy Approach	Dynamic Programming
Optimality	May not always provide an optimal solution.	Guarantees an optimal solution if the problem exhibits the principle of optimality.
Subproblem Reuse	Does not reuse solutions to subproblems.	Reuses solutions to overlapping subproblems.
Backtracking	Does not involve backtracking.	Involves backtracking.
Complexity	Typically, simpler and faster to implement.	May be more complex and slower to implement.
Application	Suitable for problems where local optimization leads to global optimization.	Suitable for problems with overlapping subproblems and optimal substructure.

Huffman Coding





Huffman Coding (2)

- The data arrive as a sequence of characters.
- Huffman's greedy algorithm uses a table giving how often each character occurs (its frequency) to build up an optimal way of representing each character as a binary string.
- Suppose that you have a 100,000-character data file. Need to store them compactly. Only 6 characters in the file occur.

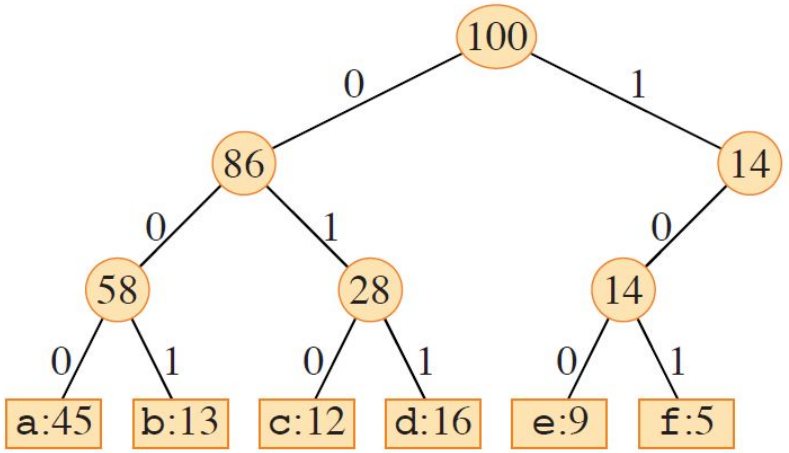
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

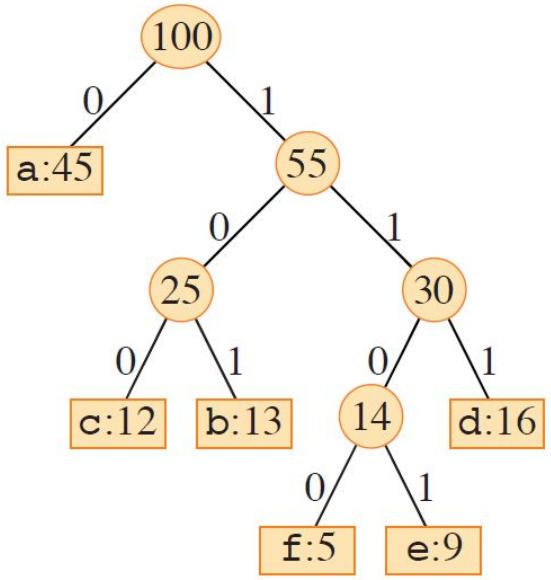
Huffman Coding (3)

- No codeword is also a prefix of some other codeword.
- “face” = 1100·0·100·1101 = 110001001101
- “bed” = 101·1101·111 = 1011101111
- “decaf” = 111·1101·100·0·1100 = 111110110001100

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Huffman Coding: Decoding





	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Huffman Coding: Cost

- Full binary tree represents an optimal code.
- Let C be the alphabet:
  then optimal tree for prefix-free code has exactly
  - $|\mathrm{C}|$ leaves and
  - $|\mathrm{C}| - 1$ internal nodes.
- Let $c.freq$ donate the frequency of symbol c in the file.
- Let $\mathrm{d_T}(c)$ donate the depth of c's leaf in the tree.
- The number of bits $B(\mathrm{T})$ needed for coding file:

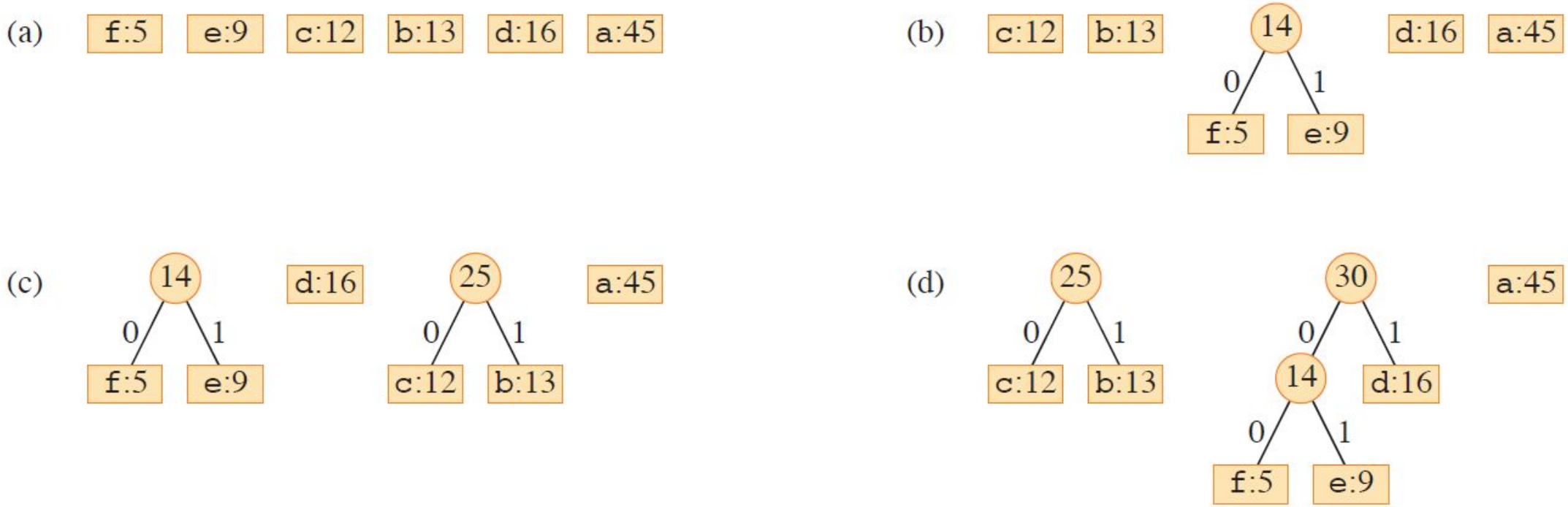
$$B(\mathrm{T}) = \sum_{\mathrm{c}\in\mathrm{C}} \mathrm{c.freq} * \mathrm{d_T}(c)$$

Huffman Coding: Algorithm

```
HUFFMAN(C)
 1  n = |C|
 2  Q = C
 3  for i = 1 to n − 1
 4       allocate a new node z
 5       x = EXTRACT-MIN(Q)
 6       y = EXTRACT-MIN(Q)
 7       z.left = x
 8       z.right = y
 9       z.freq = x.freq + y.freq
10       INSERT(Q, z)
11  return EXTRACT-MIN(Q)
```

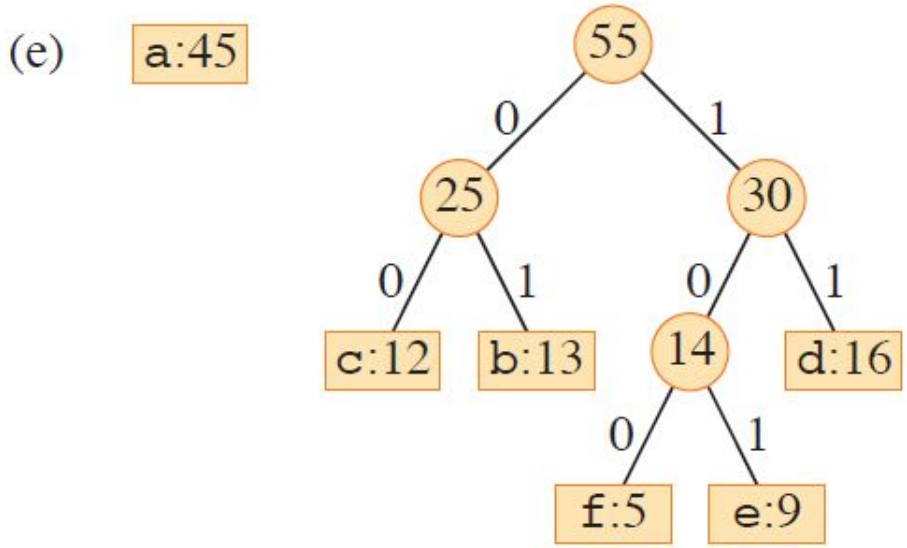
- $|C| - 1$ merging
- Q is a min heap
- $O(n * \log n)$

Huffman Coding: Building Tree (1)

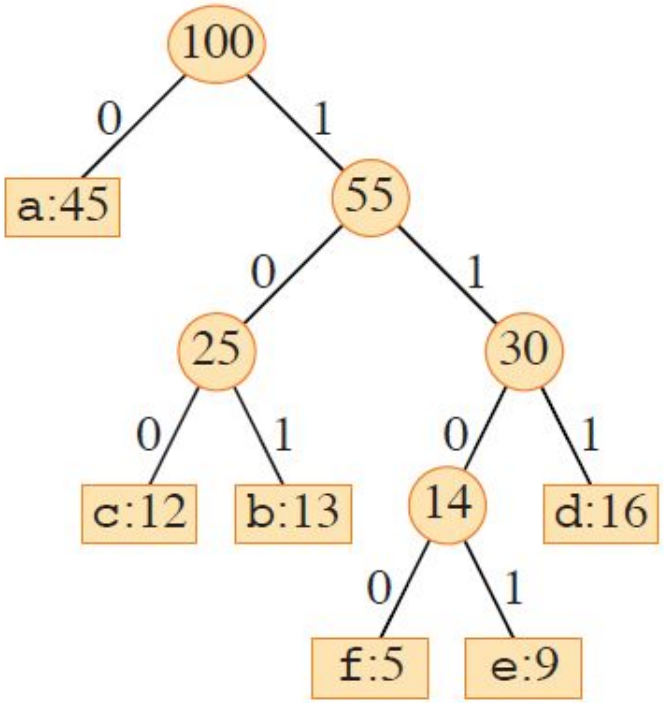


Huffman Coding: Building Tree (2)

(e)



(f)



Huffman Coding: Correctness (1)

Need to show that Huffman coding algorithms holds greedy-choice property.

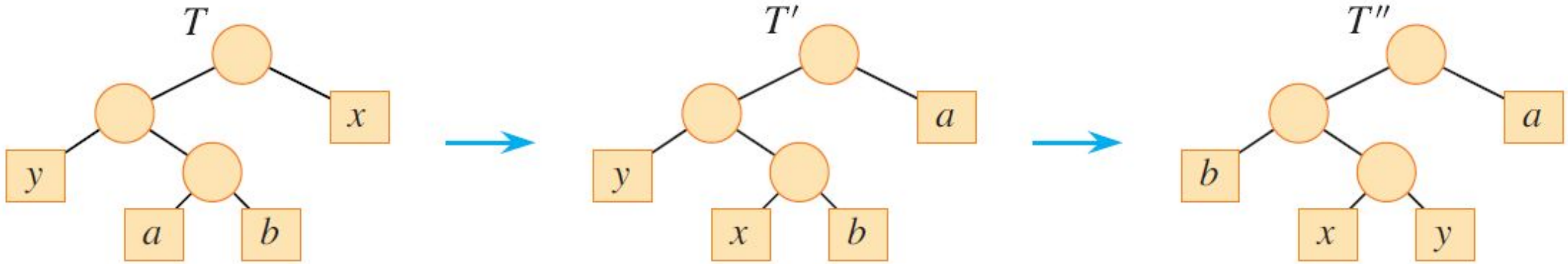
Lemma: Let C be the alphabet and each c ∈ C symbol has $c.freq$ frequency. Let x and y symbols having the lowest frequencies. Then, there exists an optimal prefix-free code for C, in which codewords for x and y have the same length and differ only in last bit.

Proof: Let a and b are sibling leaves in tree an optimal prefix-free tree T with maximum depth. Let's assume $a.freq \leq b.freq$ and $x.freq \leq y.freq$.

$x.freq \leq a.freq$ and $y.freq \leq b.freq$ as x and y have the lowest frequencies.

If $x.freq = b.freq$, then $a.freq = b.freq = x.freq = y.freq$ and lemma is proved. So, lets assume $x.freq \neq b.freq$

Huffman Coding: Correctness (2)



- $B(T) - B(T') = \sum_{c \in C} c.\text{freq} \cdot d_T(c) - \sum_{c \in C} c.\text{freq} \cdot d_{T'}(c)$

$= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a)$

$= (a.freq - x.freq) \cdot \big(d_T(a) - d_x(a)\big) \geq 0$

Similarly, $B(T'') \leq B(T') \leq B(T)$, but $B(T) \leq B(T'')$, so $\boldsymbol{B(T) = B(T'')}$

Huffman Coding: Correctness (3)

Need to show that Huffman coding algorithms holds optimal substructure property.

Lemma:

- Let C be the alphabet and each c ∈ C symbol has $c.freq$ frequency.
- Let x and y symbols having the lowest frequencies.
- Let $C' = (C - \{x, y\}) \cup \{z\}$, where for new symbol z, $z.freq = x.freq + y.freq$.
- Let T' be any tree representing an optimal prefix-free code for alphabet C'
- Then tree T, obtained from $T'$by replacing the leaf node for z with an internal node having x and y as children, represents an optimal prefix-free code for the alphabet C .

Huffman Coding: Correctness (3)

Need to show that Huffman coding algorithms holds optimal substructure property.

Proof:

- For each c $\in (C - \{x, y\})$; $\mathrm{d_T}(c) = \mathrm{d}_{T'}(c)$.
- By construction: $\mathrm{d_T}(x) = \mathrm{d}_T(y) = \mathrm{d}_{T'}(z) + 1$, then

$$B(T) - B(T') = \sum_{\mathrm{c}\in\mathrm{C}} \mathrm{c.freq} \cdot \mathrm{d_T}(c) - \sum_{\mathrm{c}\in C'} \mathrm{c.freq} \cdot \mathrm{d}_{T'}(c)$$

$$= x.freq \cdot \mathrm{d_T}(x) + y.freq \cdot \mathrm{d_T}(y) - z.\mathrm{freq} \cdot \mathrm{d}_{T'}(z)$$
$$= x.freq \cdot \mathrm{d_T}(x) + y.freq \cdot \mathrm{d_T}(x) - (x.freq + y.freq) \cdot (\mathrm{d_T}(x) - 1)$$
$$= x.freq + y.freq$$

- $B(T') = B(T) - x.freq - y.freq$

Huffman Coding: Correctness (4)

Need to show that Huffman coding algorithms holds optimal substructure property.

Proof (continue):

- Suppose that T does not represent an optimal prefix-free code for C.
- Then exists an optimal tree T' for C such that $B(T'') < B(T)$.
- Without losing generality, assume x and y are siblings in T''
- Construct T''' from T'' with replacing x and y by z: z. $freq = x.freq + y.freq$
- $B(T''') = B(T'') - x.freq - y.freq < B(T) - x.freq - y.freq = B(T')$
- This contradicts with assumption that T' represents an optimal prefix-free code for C'

Thank You